

BLEU and ROUGE are text-matching metrics used to automatically score how well an AI-generated output matches a human-written "reference" text. While originally designed for translation and summarization, they are frequently used to benchmark code generation models like [GitHub Copilot](#) and [Google Gemini Code Assist](#).

BLEU (Bilingual Evaluation Understudy)

BLEU measures precision: how many words or code snippets generated by the AI actually appear in the correct human reference text.

- How it works: It counts overlapping N-grams (sequences of N consecutive tokens or words) between the AI output and the reference.
 - The Formula: It calculates precision for unigrams (N=1), bigrams (N=2), trigrams (N=3), and 4-grams (N=4), multiplies them geometrically, and applies a penalty if the AI output is too short.
$$\text{BLEU} = \text{Brevity Penalty} \times \exp \left(\sum_{n=1}^N w_n \ln p_n \right)$$
 - Code Example: If the AI generates `return x + y;` and the human reference is `return x + y;`, the BLEU score is 1.0 (100%). If the AI generates `return y + x;`, the unigram precision is high, but higher-order bigrams fail, dropping the score.
 - Core Limit: It penalizes functional code that uses different variable names or formatting, even if the code runs perfectly.
-

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE measures recall: how much of the essential information from the human reference text was successfully captured by the AI.

- ROUGE-N: Measures the overlap of N-grams (e.g., ROUGE-1 for single words, ROUGE-2 for word pairs).
- ROUGE-L: Measures the Longest Common Subsequence (LCS). It looks for the longest structure of words that appear in both texts in the same relative order, even if they are interrupted by other words.
- The Formula (ROUGE-L Recall):
$$\text{ROUGE-L}_{\text{recall}} = \frac{\text{LCS}(\text{Reference}, \text{AI Output})}{\text{Total tokens in Reference}}$$
- Application in Code: ROUGE-L is highly useful for code evaluation because it tracks structural flow. If a human script has a specific sequence of loops and conditionals, ROUGE-L detects if

the AI kept that structural sequence, even if the AI added extra logging or comments in between.

Summary Metric Matrix

Metric Dimension	BLEU	ROUGE
Core Focus	Precision (AI output quality)	Recall (AI coverage of reference)
Primary Math	Overlapping N-grams / Total AI tokens	Overlapping N-grams / Total Reference tokens
Handling Brevity	Penalizes short outputs	Penalizes missing structural logic
Code Use Case	Best for exact line completions	Best for structural multi-line blocks

1. Calculate BLEU-1 and ROUGE-1

To see how these work mathematically, consider a single-line function generation task:

- Reference (R): `let total = price + tax;` (5 tokens)
- AI Output (O): `let total = price;` (4 tokens)

2. Compute Precision for BLEU-1

Count how many tokens in the AI output exist in the reference string.

- Tokens in O: `let` (yes), `total` (yes), `=` (yes), `price;` (yes).
- All 4 tokens match the reference.
$$\text{BLEU-1 Precision} = \frac{4}{4} = 1.0$$

3. Compute Recall for ROUGE-1

Count how many tokens from the reference string were successfully generated by the AI.

- Reference tokens captured: `let`, `total`, `=`, `price;` (4 tokens captured out of 5 total).
$$\text{ROUGE-1 Recall} = \frac{4}{5} = 0.8$$

4. Apply a Visual Graph

The behavior of structural metrics like ROUGE-L depends heavily on finding the Longest Common Subsequence (LCS) matrix. Below is a visual representation of how an LCS grid maps alignment between a Reference text and an AI Output.

		Reference Tokens					
		let	total	=	price	*	tax
AI Output Tokens	let	1	0	0	0	0	0
	total	0	1	0	0	0	0
	=	0	0	1	0	0	0
	price	0	0	0	1	0	0

Why These Metrics Fail for Advanced Code

While BLEU and ROUGE are excellent for natural language, they struggle to evaluate modern coding tools like GitHub Copilot or Gemini Code Assist accurately:

- **Functional Equivalence:** An AI might generate a `while` loop instead of a human's `for` loop. The code could be 100% functionally correct, but receive a BLEU/ROUGE score near 0 due to poor token alignment.
- **Alternative Naming:** Changing a variable name from `total` to `sum` severely damages the exact string matching scores despite zero impact on runtime.

Because of these limitations, modern AI benchmarks prefer Execution-based metrics like Pass@K (testing if the generated code actually passes unit tests) over simple text-matching matrices.

✅ Final Metric Formulation

In evaluation tasks, BLEU measures strict token generation precision, whereas ROUGE measures token and structural recall against human-verified references.

If you want to dive deeper into benchmark testing, let me know:

- Are you evaluating a custom fine-tuned model, or assessing commercial AI APIs?
- Do you want to learn how Pass@K execution testing handles code verification instead?